Valgrind Beginner's Guide

Find memory errors and leaks on Linux or WSL, with every report line explained

1. Do you need Linux?

Effectively yes, but you do not have to switch computers.

- **Linux:** works fully.
- **Windows:** Valgrind does not run natively. Use WSL2 (Ubuntu), the same setup used for g++ and GDB.
- **macOS:** support is poor and it does not work on Apple Silicon. Use ASan instead.

Install in your WSL or Linux terminal:

```
sudo apt install valgrind
```

2. What Valgrind does

Valgrind runs your program inside a virtual CPU that watches every memory access. Compared to ASan:

- **No recompiling needed.** It works on a normal binary (add `-g` to get line numbers).
- **Finds uninitialized-value reads**, which ASan cannot.
- **Much slower**, roughly 20x to 50x, so use it on small tests.

Its default tool, **Memcheck**, finds:

- Reading or writing outside heap blocks
- Use after free, double free, mismatched `new[]` / `delete`
- Use of uninitialized values
- Memory leaks, with a detailed breakdown

3. Practice program

Save as `main.cpp`. Each function has one bug; choose which to run from the command line.

```
#include <cstdlib>
#include <iostream>

void heap_overflow() {
    int* a = new int[3]{10, 20, 30};
    std::cout << a[3] << "\n";        // bug: reads past the end
    delete[] a;
}

void uninitialized() {
    int x;                            // never set
    if (x > 5) std::cout << "big\n";  // bug: depends on garbage
}

void use_after_free() {
    int* p = new int(42);
    delete p;
    std::cout << *p << "\n";          // bug: p is freed
}

void leak() {
    int* p = new int[100];            // bug: never deleted
    p[0] = 1;
}

int main(int argc, char** argv) {
    int which = (argc > 1) ? std::atoi(argv[1]) : 1;
    switch (which) {
        case 1: heap_overflow();  break;
        case 2: uninitialized();  break;
        case 3: use_after_free(); break;
        case 4: leak();           break;
    }
    return 0;
}
```

4. Compile and run

```
g++ -g -O0 main.cpp -o main
valgrind --leak-check=full --track-origins=yes ./main 1
```

- `g++ -g -O0`: debug info and no optimization. **No sanitizer flags**, because Valgrind and ASan conflict.
- `valgrind`: runs the program under Memcheck
- `--leak-check=full`: show details for every leak, not just a count
- `--track-origins=yes`: for uninitialized values, report where the value came from (slower, but very useful)
- `./main 1`: your program and its arguments

5. Reading the report

Every line starts with `==PID==` (omitted here). For case 1 you will see something like this (addresses vary):

```
Invalid read of size 4
   at 0x109192: heap_overflow() (main.cpp:5)
   by 0x109201: main (main.cpp:27)
 Address 0x4dd5050 is 0 bytes after a block of size 12 alloc'd
   at 0x484A2F3: operator new[](unsigned long)
   by 0x109170: heap_overflow() (main.cpp:4)
```

- `Invalid read of size 4`: the bug type and size. A bad write says `Invalid write`.
- `at ... (main.cpp:5)`: the exact line. The `by` lines below are the callers.
- `0 bytes after a block of size 12`: you touched the byte right after a 12-byte block (3 ints). The classic off-by-one.
- `alloc'd at ... main.cpp:4`: where that block was allocated.

Reading order: the bug type, the first line in your code, then the allocated or freed trace.

6. What each case shows

Run	Message
`./main 1`	`Invalid read of size 4` with a block-size trace
`./main 2`	`Conditional jump or move depends on uninitialised value(s)`. With `--track-origins=yes` you also get `Uninitialised value was created by a stack allocation`.
`./main 3`	`Invalid read of size 4`, plus `Address ... is 0 bytes inside a block of size 4 free'd` with the free and allocation traces
`./main 4`	A leak report at exit (next section)

7. The leak report (case 4)

```
HEAP SUMMARY:
    in use at exit: 400 bytes in 1 blocks
  total heap usage: 3 allocs, 2 frees, ... bytes allocated

400 bytes in 1 blocks are definitely lost in loss record 1 of 1
   at 0x484A2F3: operator new[](unsigned long)
   by 0x1091F0: leak() (main.cpp:20)

LEAK SUMMARY:
   definitely lost: 400 bytes in 1 blocks
   indirectly lost: 0 bytes
     possibly lost: 0 bytes
   still reachable: 0 bytes
```

- **definitely lost:** nothing points to the memory anymore. A real leak; fix these first.
- **indirectly lost:** lost only because the block that pointed to it was lost

- **possibly lost:** a pointer to the middle of a block exists; usually worth checking
- **still reachable:** memory still referenced at exit (for example globals). Usually harmless.
- `allocs` vs `frees`: if they differ, something was not freed

8. Useful options

```
valgrind --leak-check=full --show-leak-kinds=all ./main 4   # include still reachable
valgrind --error-exitcode=1 ./main                          # exit code 1 on errors (tests/CI)
valgrind --log-file=vg.txt ./main                           # write the report to a file
```

Other tools:

- `--tool=callgrind`: CPU profiling (which functions take the time)
- `--tool=massif`: heap usage over time
- `--tool=helgrind`: threading bugs

9. ASan vs Valgrind

	ASan	Valgrind
Recompile needed	Yes	No
Speed	About 2x slower	20x to 50x slower
Uninitialized reads	No	Yes
Stack array overflows	Yes	Mostly missed
Windows / macOS	Yes (Windows via WSL or MSVC)	Linux only (WSL on Windows)

Use ASan first for speed, and Valgrind as a second opinion, especially for uninitialized values.

10. Cheat sheet

What	Command
Build for Valgrind	`g++ -g -O0 main.cpp -o main`
Run with full checks	`valgrind --leak-check=full --track-origins=yes ./main`
Show all leak kinds	`--show-leak-kinds=all`
Fail on errors (CI)	`--error-exitcode=1`
Save report to file	`--log-file=vg.txt`

11. Practice tasks

1 Run all four cases and note the message and line number for each.

2 Fix case 1 (`a[2]`) and confirm the report is clean: `All heap blocks were freed -- no leaks are possible` and `ERROR SUMMARY: 0 errors`.

3 Run case 2 with and without `--track-origins=yes` and compare.

4 Add `delete[] p;` in `leak()` and confirm the leak disappears.